



**Application Project Planning
Application Project**

Students:

Ricardo Sierra Roa
Sergi Fernández Mendez

Profesor:

Thao Dang

Hochschule Esslingen
Campus Hiltop

Due Date:

December 16th, 2025

Index

Project Overview.....	4
Objectives.....	4
Hardware.....	4
Technology Stack.....	4
System Architecture (High-Level).....	5
Core Functionality.....	5
Control Strategy.....	5
Project Planning.....	6
Introduction.....	7
Week 1.....	8
Initial development of the identification system.....	8
Testing and validation of the vision system.....	9
Robot analysis and system compatibility.....	9
Initial hands-on practice with manual robot control.....	10
Week 2.....	11
Integration of the TurtleBot4 Pro camera with the development computer.....	11
Implementation of human detection on the robot.....	11
Initial rotational tracking behavior.....	11
Integration of a proportional (P) controller.....	12
Week 3.....	13
Mechanical Design and Camera Integration.....	13
Initial Linear Motion Programming.....	13
Conceptualization of Visual-Based Distance Adjustment.....	14
Week 4.....	15
Motion Analysis and Hardware Safety Constraints.....	15
Implementation of Adaptive Visual Scaling.....	15
Computational Optimization with ONNX.....	16
Week 5.....	17
Full System Integration and Onboard Deployment.....	17
Performance Benchmarking and Fluidity Analysis.....	17
Identification of Hardware Bottlenecks.....	17
Week 6.....	19
The Multi-Target Challenge and Filtering Logic.....	19
Computational Feasibility Analysis.....	19
Hardware Modification Proposal.....	19
Project Assumption: TurtleBot 4 Pro Evolution to Hybrid Architecture.....	21
Prerequisites & System Requirements for running the project.....	22
Step 1. Turtlebot 4 pro Setup.....	22
Step 2. Setup your Computer.....	26
Step 3. Run the project.....	27

Video (Project Execution Demo).....	28
Conclusions.....	29
Evolution of Objectives and Timeline.....	29
Technical Pivot: Perception and Depth Strategy.....	29
Hardware Limitations and Performance Balance.....	30
Final Project Planning.....	31
References.....	32

Project Overview

This project enables a TurtleBot4 Pro to follow a person smoothly and reliably using its onboard OAK-D Pro camera and ROS 2 Jazzy. Detection is powered by a YOLOv8 model trained in PyTorch and deployed to the camera via ONNX → OpenVINO blob, so inference runs on-device while the Raspberry Pi focuses on control and messaging.

Objectives

- Detect the person class in real time and obtain 3D position (X, Y, Z) from the OAK-D Pro.
- Track and select a single target consistently, even in crowded scenes.
- Generate smooth velocity commands to follow at a comfortable distance.
- Provide clear interfaces (topics, messages) for easy integration with navigation or logging tools.
- Keep CPU usage on the robot low by offloading inference to the camera NPU.

Hardware

- TurtleBot4 Pro (iRobot Create 3 base + Raspberry Pi 4)
- Luxonis OAK-D Pro (RGB + stereo depth + IR projector)
- Development laptop (for model training and testing)
- Power and networking accessories as required

Technology Stack

- ROS 2 Jazzy (Ubuntu 24.04) for middleware and lifecycle management.
- OAK-D Pro / DepthAI for spatial AI and depth estimation.
- YOLOv8 (Ultralytics, PyTorch) for model training and evaluation.
- ONNX + OpenVINO blob for portable, efficient on-device inference.
- Custom ROS 2 node (person_follower) for control and decision logic.

System Architecture (High-Level)

- Perception Layer: OAK-D Pro runs the compiled neural network and publishes spatial detections (bounding box + 3D coordinates + confidence).
- Tracking Layer: Maintains the selected target over time to avoid ID swaps.
- Control Layer: Computes linear and angular velocities from distance and heading errors, applying filtering, limits, and safety rules.
- Robot I/O: Velocity commands are sent to the Create 3; sensor data and status are exposed via standard ROS 2 topics.

Core Functionality

- Real-time person detection with depth-aware localization.
- Target selection and hand-off management when multiple people appear.
- Smooth motion with bounded velocities, rate limiting, deadbands, and safety stops.
- Clear, documented topics for detections, diagnostics, and velocity commands.

Control Strategy

- Distance control: PI controller to maintain a target following distance.
- Heading control: PD controller to keep the person centered in the field of view.
- Stability aids: Exponential smoothing, deadbands, anti-windup, slew-rate limits, and minimum-distance safeguards.

Project Planning

Week	Description	Objectives
1st Week	Project kickoff: define scope, risks, success metrics, milestones, and create the repository and backlog.	Approved scope and success criteria; repo + branching strategy set; prioritized backlog; dev environment ready (ROS 2 Jazzy + DepthAI).
2nd Week	Build the first perception prototype: train or select a YOLOv8 person model, export to ONNX, compile OpenVINO blob, and run it on the OAK-D to publish 3D person detections.	Real-time on-device detections (≥ 20 FPS) with XYZ published to ROS 2; basic diagnostics topic available.
3rd Week	Fine-tune and add target selection so the robot follows one specific person without confusing them with others (ID persistence, hand-off logic).	Stable single-target tracking with $< 25\%$ ID swaps in simple multi-person scenes.
4th Week	Integrate the model on-camera and start basic robot motion: connect detections to velocity commands and verify safe movement end-to-end.	Robot approaches target and respects a safe minimum distance; watchdog + loss-of-target behaviors implemented.
5th Week	Design and tune the motion controller for smooth, controlled following (PI for distance, PD for heading, with limits and filters).	Smooth /cmd_vel with bounded accel/jerk; steady following at target distance without visible oscillations; gains configurable via YAML.
6th Week	Integrate all modules: neural network, tracker, and controller, with clean interfaces and launch files.	One-command launch for full stack; topics and params documented.
7th Week	Execute tests and final fine-tuning: corridors and moderate crowds; log edge cases and adjust thresholds/models.	Track maintained $\geq 75\%$ of the time in moderate crowds; safe stop on target loss; functional test plan passed.
8th Week	Produce documentation, final report, short demo, analyze results and prepare final presentation.	Complete README + diagrams + test report; demo video recorded; backlog triaged with clear future work.

Introduction

The TurtleBot4 Pro consists of a robotic platform intended for education and research, which in its standard configuration is operated by a wireless controller and provides mobility and sensing capabilities on the robot. Building upon this foundation, the main aim of this project is to develop a system that can identify people using an intelligent vision module and enable the robot to focus on that objective and follow it autonomously. To achieve this, it has been designed a custom camera-based perception component, capable of recognizing a person in real time and assigning a similarity-based confidence score that validates the detection. Once the target is confirmed, the goal of the TurtleBot4 Pro is to maintain stable and safe follower behaviour, also it must be capable of maintaining its first assignment even though the robot has more than a person detected in the camera.

The system integrates multiple software technologies like ROS 2 Jazzy, DepthAI, and YOLOv8-based detection models, which support the human-identification camera pipeline and the interface between the robot's camera and the computer. Additionally, the TurtleBot4 Pro retains the ability to be operated via a handheld controller for exceptional manual intervention; however, this manual mode is mutually exclusive and cannot be engaged while the autonomous detection and person-following algorithms are actively executing. The resulting combination of advanced perception, autonomous control, and desktop visualization establishes a framework for developing human-robot interaction applications based on visual tracking.

Week 1

Initial development of the identification system

During the first week, the main efforts were directed toward creating the identification system that will be implemented into the TurtleBot4 Pro during the development of the project and it will work as the foundational component. The laptop's built-in camera was used during this phase, allowing rapid testing and flexible development before deploying the system to the robot.

A preliminary convolutional neural network was developed to detect humans. YOLOv8-based detection models were configured and adapted to recognize the “person” class while emphasizing facial features. Parameter tuning was performed, such as adjusting confidence thresholds, sensitivity to lighting conditions, and response to varying distances. The training performance and detection metrics obtained during this process are summarized in Figure 1.

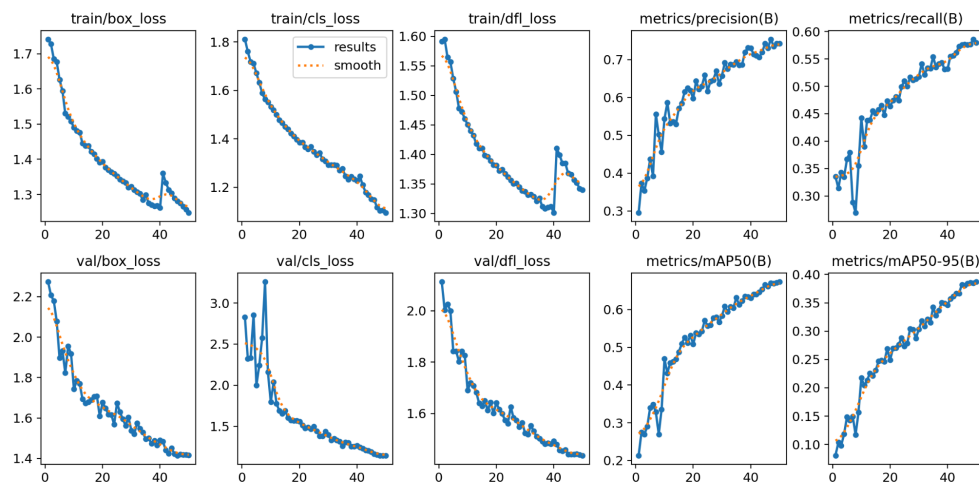


Figure 1. “Human Body Detection CNN ”

To ensure detection stability, frame-to-frame tracking techniques and filtering mechanisms were incorporated to maintain consistent recognition, even when the subject made small movements or changed posture. So, to ensure that it has been verified that if the robot detects a confidence score higher than 65% the TurtleBot4 Pro has to follow the person that is in front of the camera which in most cases should be a human.

Additionally, the conceptual design for future ROS 2 integration was prepared, including preliminary topic definitions, message structures, and communication pathways between the vision module and the robot's control node. Although this integration has not been implemented yet, but the work began with the foundational architecture of the robot for upcoming stages.

Altogether, this section represents the creation of the first and functional version of the camera that is wanted at the end of the project, also there is the definition of reliability metrics, and the technical groundwork required for the future deployment on the TurtleBot4 Pro.

Testing and validation of the vision system

In parallel with development, test sessions were conducted using the laptop camera as the experimental environment, before linking it with the robot's camera. These tests included variations in illumination, distance, deepness, and different levels of human body visibility.

The primary goal was to assess detection stability, ensure consistent human body tracking and perfectionating parameters to enhance accuracy when it comes to recognizing human people. These evaluations helped identify system limitations, refine the similarity threshold, and gather essential data for planning the migration to the robot's hardware.

Robot analysis and system compatibility

Beyond the camera and the identifying people work, a thorough analysis of the TurtleBot4 Pro was carried out to understand its internal architecture, sensors, and ROS 2 communication structure.

This review involved examining the compatibility between the TurtleBot4 Pro and the development computers, configuring the network to ensure stable connectivity, and checking the active topics and nodes to understand the robot's communication structure. It also included evaluating the built-in camera modules and reviewing the available control modes to determine how they would interact with the future autonomous functionalities.

This analysis ensured that the identification system can be integrated smoothly and that all software and hardware components will operate correctly together.

Initial hands-on practice with manual robot control

Finally, time was dedicated to operating the TurtleBot4 Pro manually using the controller. The purpose was to become familiar with the robot's physical behavior, speed, turning capabilities, stability, and responsiveness.

This initial test provides a clear intuition of how the robot is expected to move once autonomous tracking is implemented. Therefore, it is a crucial step for designing an appropriate control system in the upcoming stages.

Week 2

Integration of the TurtleBot4 Pro camera with the development computer

One of the main achievements and the main goal of this week was establishing a functional connection between the TurtleBot4 Pro camera and the computer. This integration enabled real-time visualization of the robot's camera feed on the computer screen, this advancement will streamline future development stages, simplify testing procedures, and make it easier to assess the performance of the system.

The connection was implemented through the appropriate ROS 2 camera packages, ensuring full compatibility between all the programmes used. This provided a more efficient workflow for future testing and accelerated project stages, in other words, having reached this goal the project is growing at a rapid pace.

Implementation of human detection on the robot

After completing the camera integration, the person-identification system developed in Week 1 was integrated into the TurtleBot4 Pro workflow. At this stage, the detection model was executed in its original .pt format on the development computer, while the OAK-D Pro camera was used exclusively as a sensing device to provide real-time image data. This setup enabled the robot to detect humans within the camera's field of view and assign a confidence score to each detection.

This step validated the perception pipeline and established a reliable data flow between the camera, the detection model, and the ROS 2 nodes, forming the foundation for subsequent tracking and control development.

Initial rotational tracking behavior

Another important aim of week 2 was starting with the autonomous movement of the robot when it came into identifying a person. By the moment the robot is only able to follow a person, but only with a rotational movement and keeping the target centered in the camera's field of view.

At this stage, the TurtleBot4 Pro did not move forward, backward or sideways in an autonomous way. Nevertheless, this rotational response represented a huge first step for completing the full tracking system, with this we can confirm that there is a good communication between perception, data publishing, and motion commands.

Integration of a proportional (P) controller

To enhance the precision and stability of the robot's rotational movement, a proportional (P) controller was implemented. This controller is responsible for the angular velocity based on the horizontal error between the detected person's position and the center of the image, resulting in smoother rotations, for increasing the safety of the robot and people around.

This integration laid the groundwork for the full control strategy to be developed in later weeks, including additional controllers and safety mechanisms.

Week 3

Mechanical Design and Camera Integration

This week marked the transition from software simulation to physical hardware integration. The primary task was the design and fabrication of a custom support bracket for the OAK-D Pro camera. This structural modification was essential because the standard mounting position available on the robot was situated too low, restricting the camera's focus solely to the feet of the subjects. To overcome this limitation, the new support was specifically designed to elevate the sensor, ensuring the best possible viewing angle to capture the full human figure rather than just the lower extremities. Using 3D printing technology, a mount was prototyped to attach the camera securely to the TurtleBot4 Pro's existing mounting points.

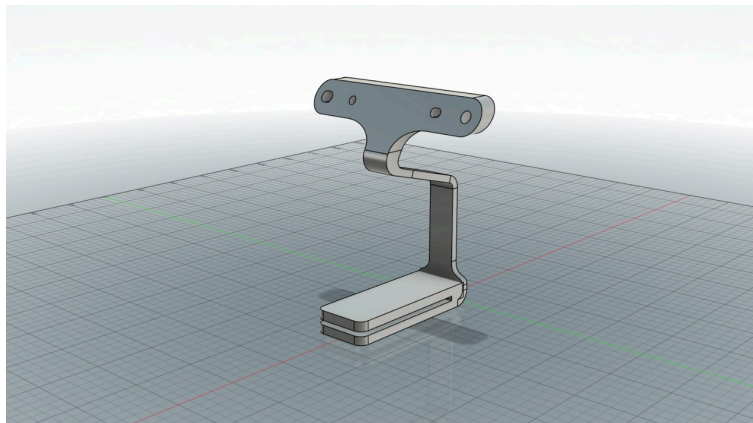


Figure 2. "Custom 3D camera bracket"

Once printed, the support was installed on the robot to verify the structural integrity and, more importantly, the viewing angle. Extensive testing was conducted to ensure the camera's tilt provided an optimal field of view, capable of detecting humans at various distances without cutting off essential features like feet or faces, which are critical for the depth estimation algorithms.

Initial Linear Motion Programming

With the perception hardware in place, development shifted to the robot's linear mobility. The objective was to program the robot to move forward and backward based on the distance to the target. The control logic aimed to translate the depth data received from the camera into linear velocity commands (`cdm_vel.linear.x`), initiating the first tests of longitudinal

following behavior. However, once we began working on this implementation, we realized that programming backward movement was impossible due to the robot's hardware safety configuration. The TurtleBot4 Pro is equipped with sensors (bumpers) exclusively on the front, which protect it from damage during forward motion. In contrast, the rear section lacks any type of sensor coverage. Consequently, to ensure the safety of the hardware and prevent potential collisions, falls, or structural damage, backward movement is strictly restricted. Therefore, the tracking system was finalized to follow the target exclusively using forward movements and rotations on its own axis.

Conceptualization of Visual-Based Distance Adjustment

Towards the end of the week, alongside the mechanical testing, we began to address a software challenge related to detection accuracy. We observed that the YOLOv8 model's confidence dropped significantly when the subject was too far away, as the pixel density of the person was insufficient for reliable feature extraction. To solve this, we conceptualized a logic based on "Adaptive Visual Scaling." The core idea initiated this week is to use the apparent size of the person in the camera frame as a dynamic input for the control system, aiming to program the robot to automatically seek the optimal distance where detection reliability is maximized.

Week 4

Motion Analysis and Hardware Safety Constraints

Testing the linear motion revealed a significant hardware limitation regarding backward movement. While the robot successfully executed forward commands, the backward motion was consistently blocked by the robot's internal safety protocols. It was determined that the iRobot Create 3 base restricts blind backward movement to prevent collisions, as it lacks rear-facing obstacle sensors in this configuration. Consequently, the control strategy was adapted to prioritize forward following, utilizing the front bumper sensors to handle close-proximity safety. This ensured that while the robot could not retreat autonomously, it could safely stop and navigate forward without risking collisions with unseen obstacles behind it.

Implementation of Adaptive Visual Scaling

Building upon the concept initiated in Week 3, a robust control algorithm was developed to optimize the robot's positioning based on the subject's visual footprint. We implemented a "Visual Servoing" closed-loop controller that utilizes the dimensions of the detection bounding box directly from the OAK-D Pro inference results.

The technical implementation relies on a proportional control strategy where the "error" is defined as the difference between the current height of the detected person (in pixels) and a target setpoint (e.g., the subject should occupy 60% of the frame's vertical height).

- **Approaching Logic:** When the bounding box height is below the target threshold, it indicates the subject is far away and perception resolution is low. The controller generates a positive linear velocity command proportional to this error, driving the robot closer to increase the pixel density of the target.
- **Optimal Positioning:** As the robot approaches and the bounding box grows to meet the target setpoint, the velocity command decreases, stabilizing the robot at the ideal distance for high-confidence detection.

This logic works in tandem with the previously defined "Safety Stop." While the Visual Servoing attempts to close the gap to improve "sight," the safety layer overrides this command if the physical distance (measured by depth or proximity) breaches the safety limit, ensuring the robot never compromises safety for the sake of better vision.

Computational Optimization with ONNX

To prepare for standalone operation, the software stack underwent a critical optimization phase. The YOLOv8 model, originally trained and run in PyTorch (.pt format) on the development laptop, was converted into the ONNX (Open Neural Network Exchange) format. This conversion was a prerequisite for compiling the model into an OpenVINO blob, allowing it to run directly on the OAK-D's internal Myriad X VPU (Vision Processing Unit). This step was essential to offload the heavy computational burden of neural network inference from the host computer, reducing the processing power required for the next stage of on-robot deployment.

Week 5

Full System Integration and Onboard Deployment

Following the software optimization phase, this week focused on the complete integration of all developed modules, including perception, visual servoing, and safety logic, within the final system architecture. An optimized OpenVINO blob deployment on the OAK-D Pro was initially explored; however, due to configuration complexity, hardware limitations, and project time constraints, this approach was not adopted. Instead, the final configuration relied on executing the perception pipeline using an ONNX model running on an external computer with GPU acceleration, while the ROS 2 control nodes were launched on the TurtleBot4 Pro's internal Raspberry Pi 4. The primary objective was to validate the “end-to-end” operation of the system, ensuring reliable target detection and tracking behavior. Functional tests confirmed that the robot successfully identified the target and executed the tracking logic, effectively combining rotational centering with forward approach movements defined in previous weeks.

Performance Benchmarking and Fluidity Analysis

While the robot successfully performed the tracking task, a rigorous performance analysis revealed a critical limitation regarding overall system fluidity. When attempting to execute the full perception and control pipeline onboard, the system exhibited noticeable latency between target detection and the execution of the corresponding velocity commands. This delay resulted in a non-continuous, “stop-and-go” behavior rather than the smooth and natural following motion originally intended. In contrast, offloading the perception inference to an external GPU-enabled system significantly reduced these delays, leading to more stable and responsive tracking behavior.

Identification of Hardware Bottlenecks

A diagnostic review of the system resources determined that the default onboard hardware configuration, specifically the Raspberry Pi 4, is insufficient to handle the concurrent computational load of the complete application stack. High-bandwidth ROS 2 message passing, image processing for visual servoing, and real-time coordinate transformations

impose a processing overhead that introduces delays in the control loop. These findings motivated the decision to offload the perception workload to an external computing unit, confirming that while the onboard hardware is adequate for basic demonstrations, it lacks the computational capacity required to achieve a truly fluid and responsive autonomous following experience.

Week 6

The Multi-Target Challenge and Filtering Logic

With the robot capable of following humans, the final development phase focused on a critical refinement: Target Selectivity. In its current state, the YOLOv8 model detects the class "person" generically. This means the robot identifies *all* individuals within its field of view indiscriminately. In scenarios with multiple people, the control logic struggles to distinguish the primary user from bystanders, leading to potential "ID swaps" where the robot might mistakenly begin following the wrong person if they cross paths. To address this, we analyzed the implementation of an advanced filtering layer. The objective was to modify the Convolutional Neural Network (CNN) pipeline or integrate a secondary "Re-Identification" (Re-ID) algorithm. This would theoretically allow the system to extract specific feature vectors (tags) from the target such as clothing color or specific facial features to lock onto a single individual and ignore others.

Computational Feasibility Analysis

Upon attempting to design and integrate this secondary filtering layer, we encountered an insurmountable hardware barrier. As determined during the testing in Week 5, the TurtleBot4 Pro's internal Raspberry Pi 4 was already operating near saturation just to maintain the basic detection and motion control loops. Consequently, adding the necessary logic to filter the image, process additional neural network tags, or run a robust tracking algorithm (such as DeepSORT) requires a significant increase in floating-point operations per second (FLOPS) that the current system cannot provide. Our analysis demonstrated that enabling any form of advanced pixel-level filtering caused the system latency to spike to unacceptable levels, frequently exceeding one second. This delay rendered the robot effectively uncontrollable, proving that the current onboard hardware simply lacks the computational headroom to process the identity-based "who is who" logic while simultaneously handling the spatial "where is it" calculations.

Hardware Modification Proposal

Confronted with the impossibility of implementing fluid single-target tracking on the stock hardware, we formulated a definitive technical proposal for the system's evolution, concluding that software optimization alone has reached its absolute limit. Consequently, we operate under the assumption that to achieve the project's ultimate goal of fluid, specific

person-following, the robot's computing core must be upgraded. Specifically, we propose replacing or augmenting the Raspberry Pi 4 with an NVIDIA Jetson Nano or a similar edge AI accelerator. Unlike the CPU-bound Raspberry Pi, the Jetson's dedicated GPU cores would be capable of handling the heavy lifting of image filtering and complex CNN execution, thereby allowing for the real-time processing of specific tags required for target locking. Furthermore, to support the high data throughput necessary for debugging and video streaming without lag, we identified the critical need for a high-performance router capable of 500MB/s transmission. This hardware overhaul represents the identified path to bridging the gap between the current functional prototype and a fully fluid, robust application.

Project Assumption: TurtleBot 4 Pro Evolution to Hybrid Architecture

Based on the experimental bottlenecks identified during the final testing phases, we formally posit that the current computational architecture, reliant solely on the standard Raspberry Pi 4, is insufficient for the project's advanced requirements. Consequently, we assume that evolving the system into a Hybrid Computing Architecture is the mandatory requirement to achieve fluid, commercial-grade autonomy. This proposal involves augmenting the robot's hardware by integrating an NVIDIA Jetson Nano (or superior Edge AI accelerator) to work in tandem with the Raspberry Pi.

While the Raspberry Pi continues to handle low-level tasks such as motor control and sensor drivers, the Jetson would take over the high-level perception pipeline. Unlike the CPU-bound processing of the current setup, the Jetson provides approximately 472 GFLOPS of parallel compute power via its dedicated CUDA cores. This shift would drastically reduce inference latency from the current >500ms to near real-time performance (<30ms), enabling the implementation of complex "Target Locking" algorithms that are currently impossible. Furthermore, to support the massive data throughput required for uncompressed video debugging and sensor fusion without saturating the internal bus, a high-performance router capable of sustained 500MB/s transmission is critical. This hardware overhaul represents the most technically sound and cost-effective path to bridge the gap between the current functional prototype and a fully robust, autonomous application.

Prerequisites & System Requirements for running the project

The following prerequisites are required in order to run the project:

- A custom 3D-printed mounting base for attaching the camera to the upper part of the robot. [Link to the 3D Model](#).
- A computer running Ubuntu 24.04 LTS with ROS 2 Jazzy properly installed and configured.
- A Turtlebot 4 Pro platform.
- An OAK-D camera connected to the Turtlebot for real-time visual data acquisition.
- A router capable of providing a stable wireless connection between the computer and the Turtlebot.
 - The router must support a data transfer rate of at least 500 Mbps, due to the continuous transmission of video data from the OAK-D camera to the laptop for real-time processing.
- (Recommended) A computer equipped with a GPU, with the appropriate drivers and CUDA toolkit installed.
 - While not strictly required, GPU acceleration significantly improves the performance and smoothness of the CNN-based processing.

Step 1. Turtlebot 4 pro Setup

- 1) Connect the Turtlebot 4 Pro into the dock station (as in *figure. 3*), so it's turn on and battery must be at least 30%

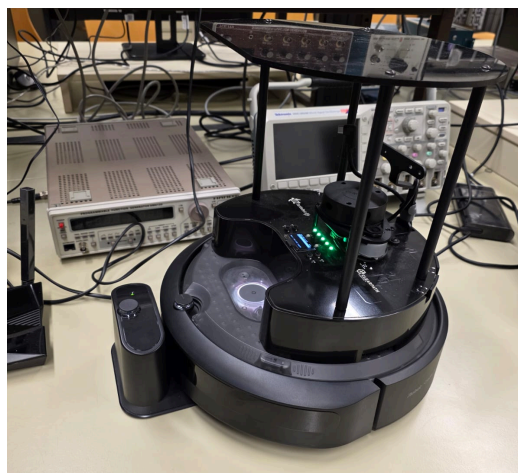


Figure 3. "Connect Turtlebot 4 to dock station"

- 2) After connecting the Turtlebot to the docking station, it will power on automatically. A wireless network with the SSID: Turtlebot4 will appear on your computer. Connect to this network using the password: Turtlebot4

- 3) Open a terminal and connect to the Turtlebot via SSH using the following command:

```
Shell
ssh ubuntu@10.42.0.1
```

Note: You will be asked for a password, the default password is turtlebot4. When the robot is brand new, the default IP address is 10.42.0.1. If the robot has been previously configured, the current IP address can be found on the Turtlebot's screen (see Figure 4).



Figure 4. “IP Adress”

- 4) After connecting via SSH, run the following command to configure the Turtlebot network:

```
Shell
turtlebot4-setup
```

This command will open the network configuration interface shown in Figure 5.

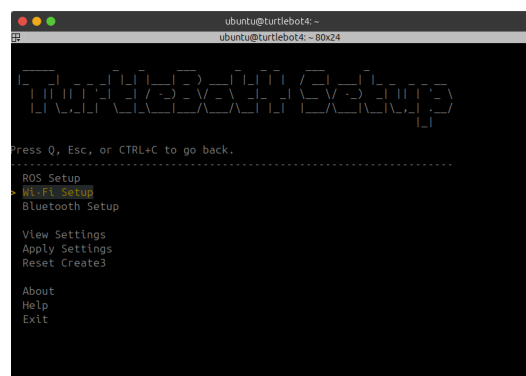


Figure 5. “Connect turtlebot to network”

Select the Wi-Fi Setup option. The network configuration menu shown in *Figure 6* will appear.

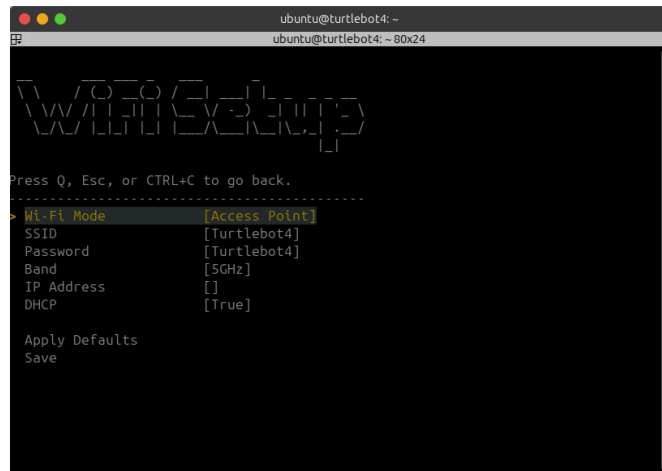


Figure 6. “Network configuration”

Set the network mode to Wi-Fi: Client. Enter the SSID and password of the router specified in the prerequisites. For demonstration purposes, the following example is shown:

- SSID: Turtlebot4Network
- Password: turtlebot4

Note: Do not use these example credentials. Always enter your own router’s SSID and password.

The configuration process is illustrated in Figure 7.

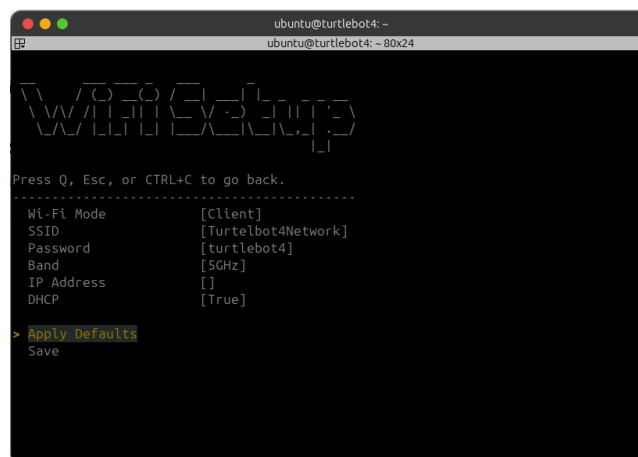


Figure 7. “Network configuration modification”

Click Save to apply the changes. When prompted, enter the user password: turtlebot4.

Finally, click Apply Settings, as shown in Figure 8.

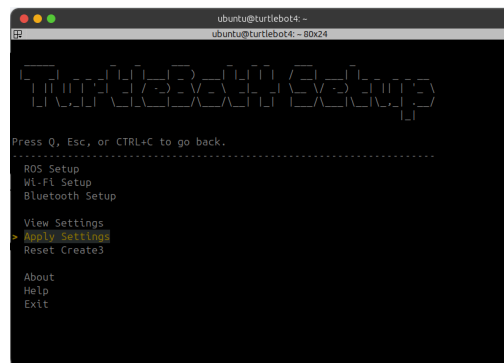


Figure 8. “Apply Settings”

Confirm the changes by clicking Yes, as shown in Figure 9.

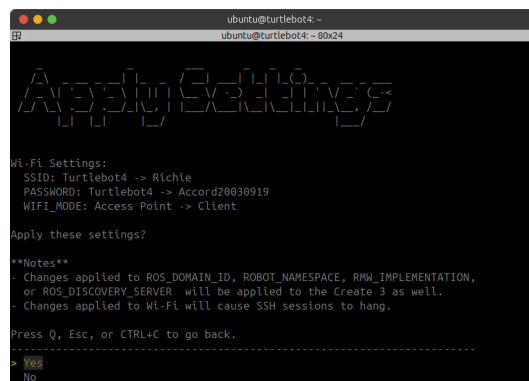


Figure 9. “Save Settings”

After applying the configuration, a message with warnings might appear in the terminal, as shown in Figure 10..

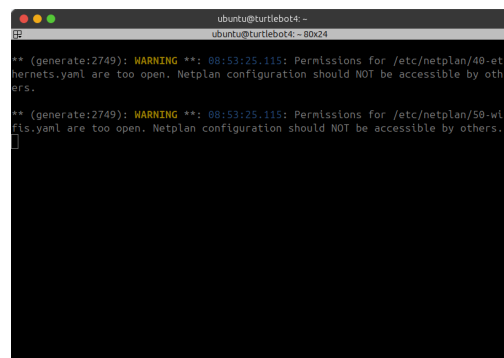


Figure 10. “Finish network configurations”

Application Project

At this point, the Turtlebot will attempt to connect to the configured router. Once connected, the robot will be ready to run the project. If the connection is not established immediately, power off the Turtlebot by pressing the large round button. Then, place it back on the docking station to power it on again. After rebooting, the robot should automatically connect to the router.

Note: Once connected to the router, the Turtlebot's IP address will change.

Step 2. Setup your Computer

- 1) Ensure that your computer has an active internet connection, then clone the following GitHub repository:

```
Shell
git clone https://github.com/Ricardo0919/turtlebot-4-pro-trackerProject.git
```

Note: For the next steps, ensure that ROS 2 Jazzy and rqt_image_view are installed, pip dependency of python as they are required for running the project and visualizing the video stream.

- [Installation of Ros2 Jazzy Ubuntu 24.04](#)

For rqt_image_view

```
Shell
sudo apt update
sudo apt install ros-jazzy-rqt -y
```

- 2) Next, install the Python dependencies listed in requirements.txt. Navigate to the root directory of the cloned repository and run the following command:

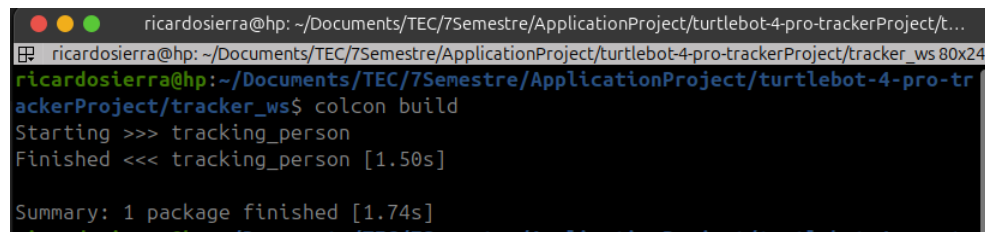
```
Python
python3 -m pip install --break-system-packages -r requirements.txt
```

Application Project

- 3) After installing the requirements, navigate to the `tracker_ws` folder (*workspace*) and run the following commands:

```
Shell
source /opt/ros/jazzy/setup.bash
colcon build
```

Note: After completing the build process, you should see an output similar to the one shown in Figure 11. Do not close this terminal window, as it will be used later.



```
ricardosierra@hp: ~/Documents/TEC/7Semestre/ApplicationProject/turtlebot-4-pro-trackerProject/t...
ricardosierra@hp:~/Documents/TEC/7Semestre/ApplicationProject/turtlebot-4-pro-trackerProject/tracker_ws 80x24
ricardosierra@hp:~/Documents/TEC/7Semestre/ApplicationProject/turtlebot-4-pro-trackerProject/tracker_ws$ colcon build
Starting >>> tracking_person
Finished <<< tracking_person [1.50s]

Summary: 1 package finished [1.74s]
```

Figure 11. “Output Colcon Build”

Step 3. Run the project

- 1) Ensure that both the TurtleBot and your computer are connected to the same router network. In this example, the network SSID was Turtlebot4Network.
- 2) Open a new terminal window and connect to the TurtleBot via SSH using the following command. Replace the IP address with your own:

```
Shell
ssh ubuntu@192.160.0.6
```

Note: You will be asked for a password, the default password is turtlebot4.

- 3) In the terminal where you are logged into the TurtleBot, run the following command to enable and use the robot’s camera:

```
Shell
ros2 launch turtlebot4_bringup oakd.launch.py
```

After the command finishes, the message “Loaded node ‘/oakd’ in container ‘oakd_container’” will appear in the terminal. Once this message is displayed, press Ctrl + C.

- 4) Now, in the same terminal where colcon build was previously executed, run the following commands:

```
Shell
source install/setup.bash
ros2 launch tracking_person Tracker_launch.py
```

After running the commands, two rqt_image_view windows should appear, as shown in *Figure 12*.

- These windows allow you to directly visualize the image streams being processed.
- The image published on the topic /oakd/rgb/preview/image_raw corresponds to the raw image directly captured from the camera.

The image published on the topic /tracking_person/annotated corresponds to the processed image used for person tracking.

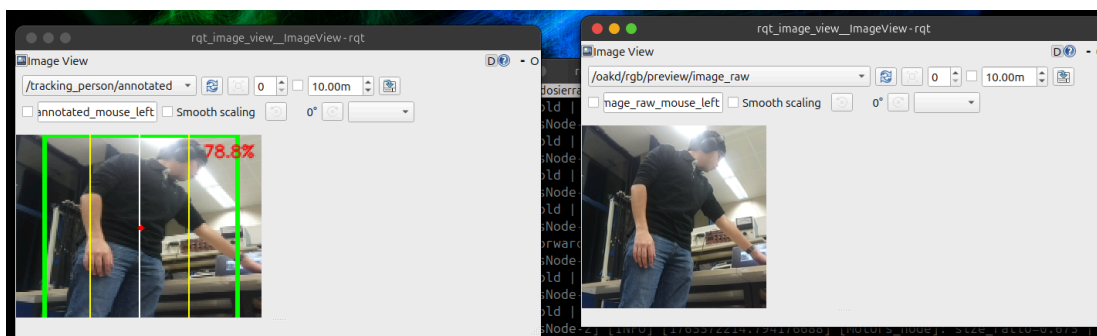


Figure 12. “Visualization of camera and tracking output”

- 5) If you have reached this point, you have successfully set up and executed the project.

Video (Project Execution Demo)

The following video demonstrates the successful execution of the project, showcasing the TurtleBot 4 Pro performing real-time person tracking and following based on visual detection.

Link: <https://youtu.be/b1HFeXYkS2I>

Conclusions

The completion of this Application Project has successfully transformed a standard educational platform, the TurtleBot4 Pro, into a robotic agent capable of real-time person following. While the fundamental objective of identifying and tracking a human subject has been achieved, the development process revealed a significant disparity between the initial theoretical objectives and the technical reality imposed by the hardware and software integration.

At the outset, the project scope was defined under an idealistic perspective, stemming from a lack of prior experience with the specific constraints of the iRobot architecture and the Luxonis deployment workflow. This learning curve became the determining factor that redefined the final capabilities of the prototype, shifting the focus from a purely 3D omnidirectional navigation system to an optimized, constraint-based solution.

Evolution of Objectives and Timeline

It is important to note that the objectives established at the beginning of the project have undergone slight modifications throughout the process. When originally drafted, the specific limitations of the robot were unknown, and we encountered a lack of specific resources necessary to fully resolve certain initial goals. Additionally, regarding the schedule, the project was originally planned for an 8-week duration; however, through efficient execution and scope adaptation, we successfully completed the development in just 6 weeks.

Technical Pivot: Perception and Depth Strategy

One of the most significant engineering deviations occurred within the perception layer. Although the OAK-D Pro hardware is equipped with stereo sensors and an IR projector for full 3D depth extraction, our final implementation relies exclusively on 2D vision.

The adopted workflow involved training a custom neural network (YOLOv8), exporting it to ONNX, and subsequently compiling it into a BLOB format for execution on the camera's Myriad X VPU. During this process, we discovered that synchronizing the extraction of raw

depth data (disparity maps) alongside the custom CNN inference within the same OpenVINO pipeline required complex internal node configurations that exceeded the project's timeframe.

Faced with this integration barrier, we decided to bypass the hardware depth sensor. Instead, we engineered a solution based on projective geometry (a "rule of three"). By establishing a proportional relationship between the pixel height of the detected bounding box and the physical distance of the subject, we enabled the robot to estimate proximity using only 2D data. This abstraction allowed for effective distance control without the computational and configurational overhead of processing the 3D point cloud.

Hardware Limitations and Performance Balance

The resulting system is functional and fulfills the core requirement of following a person, but it operates within a strict "safety window" defined by the hardware capabilities:

1. **Motion Constraints:** The absence of rear-facing sensors on the Create 3 base necessitated a hard restriction on the control strategy. To guarantee hardware integrity, the tracking logic was limited to forward movements and on-axis rotations, sacrificing bidirectional maneuverability for safety.
2. **Computational Bottleneck:** Empirical testing confirmed that the Raspberry Pi 4 acts as a significant bottleneck. Despite offloading the vision processing to the camera, the management of ROS 2 communication and control logic saturates the CPU. This prevents the implementation of advanced "Target Locking" (ID persistence) and results in non-fluid motion latency.

In summary, this project serves as a successful proof of concept that validates the software logic and the visual servoing strategy. However, we conclude that to achieve a robust final product capable of utilizing native 3D depth and discriminating specific users with fluid motion, it is imperative to migrate to the proposed Hybrid Computing Architecture (integrating an NVIDIA Jetson Nano), thereby removing the processing barriers encountered during this development.

Final Project Planning

Week	Description	Objectives
1st Week	Initial Development & Analysis: Developed YOLOv8 person detection using laptop camera. Analyzed TurtleBot4 architecture and communication nodes.	Foundation: Validated detection with >65% confidence. Verified system compatibility and manual control.
2nd Week	Integration & Rotation: Connected OAK-D Pro to ROS 2. Deployed YOLOv8 .pt detection running on the development computer. Implemented P-controller for rotational alignment.	Basic Tracking: Real-time onboard detection achieved. Robot successfully rotates to keep the target centered.
3rd Week	Hardware & Motion Strategy: Designed 3D-printed mount for better camera angle. Attempted linear motion and discovered rear-sensor safety locks.	Pivot: Camera angle optimized. Motion strategy redefined to forward-only due to hardware safety constraints.
4th Week	Visual Servoing & Optimization: Implemented "Adaptive Visual Scaling" (distance based on bounding box height). Compiled model to BLOB for Myriad X VPU.	Optimization: Bypassed complex 3D depth for efficient 2D projective geometry. Offloaded inference to camera to save CPU.
5th Week	Full Integration & Benchmarking: Deployed full "end-to-end" autonomy onboard. Analyzed motion fluidity and system resources.	Bottleneck Identified: Validated autonomous following. Confirmed Raspberry Pi 4 CPU saturation causes latency and "jerky" motion.
6th Week	Multi-Target Analysis & Conclusion: Attempted ID-filtering logic. Analyzed computational limits for specific user tracking.	Final Verdict: Proof of concept completed. Determined that Hybrid Architecture (Jetson Nano) is required for fluid, multi-target tracking.

Github Repository

Link to the github repository:

<https://github.com/Ricardo0919/turtlebot-4-pro-trackerProject>

References

- Clearpath Robotics. (2023). “*TurtleBot 4 user manual*”. TurtleBot4. Retrieved November 28, 2025, from: <https://turtlebot.github.io/turtlebot4-user-manual/>
- Clearpath Robotics. (2025). *TurtleBot 4 software & tutorials*. In *TurtleBot 4 User Manual*. Retrieved November 28, 2025, from https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_common.html
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
- Luxonis. (2024). *DepthAI API documentation*. Retrieved November 28, 2025, from <https://docs.luxonis.com/software/>
- Luxonis. (2024). *OAK-D Pro technical specifications*. Retrieved November 28, 2025, from: <https://docs.luxonis.com/hardware/products/OAK-D%20Pro>
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66). <https://doi.org/10.1126/scirobotics.abm6074>
- NVIDIA Corporation. (2019). *Jetson Nano developer kit user guide*. <https://developer.nvidia.com/embedded/jetson-nano-developer-kit>
- Open Robotics. (2024). *ROS 2 CLI tools and client libraries*. In *ROS 2 Documentation*. Retrieved November 28, 2025, from <https://docs.ros.org/en/jazzy/Tutorials.html>
- Open Robotics. (2024). *ROS 2 Jazzy Jalisco* [Middleware software]. <https://docs.ros.org/en/jazzy/>
- Ultralytics. (2023). *YOLOv8 modes: Export and predict*. Retrieved November 28, 2025, from <https://docs.ultralytics.com/modes/export/>